Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

Develop a lighttpd test setup for ECH

Seán Joseph Lawlor

A Report

Presented to the University of Dublin, Trinity College

in partial fulfilment of the requirements for the degree of

BA(Mod) in Science in Computer Science

Supervisor: Dr. Stephen Farrell

May 2026

Develop a lighttpd test setup for ECH

Seán Joseph Lawlor, BA(Mod) in Science in Computer Science

University of Dublin, Trinity College, 2026

Supervisor: Dr. Stephen Farrell

This project explores the development of a local test environment for evaluating Encrypted Client Hello (ECH) using the Lighttpd web server. ECH, an extension to TLS 1.3, encrypts sensitive information in the ClientHello message to improve user privacy. The goal of this report is to build, configure, and test a working Lighttpd installation capable of performing ECH handshakes using BoringSSL and to verify its functionality through practical experimentation. The work includes compiling Lighttpd with ECH-capable TLS libraries, generating ECH configuration files, and validating results with tools such as `bssl client`, `curl` and modern browsers. The outcome demonstrates the feasibility of integrating ECH within lightweight web server environments and documents configuration issues, debugging approaches, and performance observations.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Dr. Stephen Farrell, for his guidance, support, and valuable feedback throughout the course of this project. His expertise and direction were instrumental in shaping both the implementation and evaluation presented in this work.

Seán Joseph Lawlor

University of Dublin, Trinity College
May 2026

Contents

List of Tables

List of Figures

Chapter 1

Introduction

Encrypted ClientHello (ECH) is an emerging extension to the TLS 1.3 protocol designed to protect sensitive metadata during the TLS handshake. While TLS provides strong encryption for application data, the initial ClientHello message remains unencrypted. This exposes information such as the Server Name Indication (SNI) and Application-Layer Protocol Negotiation (ALPN). The visibility of this metadata enables network observers to monitor, profile, censor, or otherwise interfere with user activity, even when the underlying communication is encrypted.

ECH addresses this limitation by encrypting the ClientHelloInner message using Hybrid Public Key Encryption (HPKE). This conceals the intended destination domain and significantly enhances privacy at the network layer.

Lighttpd is a lightweight, high-performance, open-source web server designed for environments where efficiency, low memory usage, and simplicity are essential. Unlike larger platforms such as Apache or Nginx, Lighttpd prioritises a minimal footprint while maintaining support for modern web standards through a modular architecture. Its event-driven design enables it to handle large numbers of concurrent connections efficiently, making it well-suited to experimental and resource-constrained environments.

Despite its potential, ECH remains in an experimental stage and is not widely deployed or tested. There is currently a lack of reproducible environments and structured methodologies for evaluating ECH-enabled systems. This project addresses that gap by designing and implementing a test environment for ECH using a Lighttpd-based web server. The work focuses on enabling ECH functionality, constructing a testing framework, and analysing protocol behaviour under different configurations.

1.1 Motivation

Metadata leakage remains a critical weakness in otherwise secure communications. The SNI field in TLS has become a significant point of exposure, enabling censorship, traffic analysis, and user tracking. For organisations concerned with privacy, including civil society groups, journalists, and human rights organisations, mitigating SNI-based surveillance is essential.

ECH provides a promising solution, but its deployment introduces several challenges. These include reliance on experimental TLS libraries, additional DNS configuration requirements, and limited tooling support. Furthermore, there is a lack of clear documentation outlining how ECH-enabled servers should be configured and evaluated in practice.

The motivation for this project is therefore twofold:

- To create a reproducible ECH testing environment suitable for experimentation, debugging, and educational use.
- To evaluate Lighttpd's experimental ECH implementation, documenting its behaviour, limitations, and compatibility with BoringSSL tooling.

By addressing these challenges, this work aims to lower the barrier to ECH experimentation and support future research and deployment.

1.2 Project Objectives

The central research question guiding this work is:

> *How can Encrypted ClientHello be enabled, tested, and validated on a Lighttpd web server in an efficient and reproducible manner?*

This question is explored through the following objectives:

- **Understand the ECH protocol and associated technologies.** This includes studying TLS 1.3, HPKE, DNS requirements, and the structure of ClientHelloInner and ClientHelloOuter messages, supported by practical experimentation with BoringSSL.
- **Compile and configure Lighttpd with ECH support.** This involves building Lighttpd from source, linking it against BoringSSL, configuring ECH key material, and resolving integration challenges.

- **Develop an automated ECH test harness.** Initial testing utilises the `bssl client` tool to inspect handshake behaviour, followed by automation to ensure repeatability and structured output.
- **Analyse TLS handshake behaviour.** This includes interpreting negotiation states, debugging errors such as `TLSV1_ALERT_ECH_REQUIRED`, and classifying outcomes as accepted, rejected, or fallback.
- **Develop automated command-line based testing.** Testing is performed using tools such as `bssl client` and `curl` to analyse TLS handshake behaviour and validate HTTP responses in a controlled and reproducible manner.

A biweekly timeline of the project is shown in Figure 1.1.

1.3 Research Contributions

This work makes the following contributions:

- **A reproducible environment for building and testing ECH on Lighttpd.** This includes a complete toolchain for compiling BoringSSL, integrating it with Lighttpd, and generating ECH key material.
- **A structured analysis of ECH handshake behaviour.** The project documents negotiation states, error conditions, and observable behaviours during testing.
- **An evaluation of Lighttpd's experimental ECH implementation.** This includes identifying limitations, configuration challenges, and interoperability issues.
- **An automated testing framework for ECH evaluation.** The project develops a structured testing approach using command-line tools to validate handshake behaviour, classify ECH negotiation outcomes, and ensure reproducibility of results.

The methodology developed in this work provides a foundation for future research into ECH deployment, interoperability testing, and secure web communication.

Period	Goals / Deliverables	Notes
Oct 14–27	- Begin learning LaTeX basics - Set up Ubuntu, build tools, Git	Focus on environment setup and version control
Oct 28–Nov 10	- Study ECH protocol - Test basic TLS setup and certificates - Research/testing strategy - Document initial setup in LaTeX	Start simple diagrams or notes
Nov 11–24	- Enable ECH on Lighttpd - Capture test results with `bssl` / `openssl` - Upload results to GitHub	Achieve a working proof-of-concept
Nov 25–Dec 8	- Test ECH using Chrome, Firefox, etc. - Introduce Playwright for automated headless testing - Validate ECH behaviour in real browsers - Update LaTeX documentation	Include screenshots and command outputs
Dec 9–22	- Basic performance measurements - Expand Playwright tests (timing, errors) - Begin structured LaTeX report	Prepare graphs/charts
Dec 23–Jan 5	- Mid-project review and summary - Update GitHub with configs and Playwright code - Produce short progress PDF	
Jan 6–19	- Refine ECH setup and tests - Improve Playwright traces, logs, screenshots - Fix server or test issues	Keep repo updated
Jan 20–Feb 2	- Finalise ECH implementation on Lighttpd - Automate recurring Playwright tests (CI-ready)	Begin organising final report
Feb 3–16	- Run comprehensive ECH tests - Compare `bssl client` vs Playwright behaviour - Draft full LaTeX report	Include methodology + analysis
Feb 17–Mar 1	- Review and refine LaTeX report - Add figures, tables, diagrams, browser traces	Ensure clarity and reproducibility
Mar 2–15	- Supervisor feedback and revisions - Finalise Playwright scripts and documentation	Clean up repo + report
Mar 16–29	- Prepare presentation/demo - Test demo setup (Lighttpd + Playwright interaction)	Keep slides concise and reproducible
Mar 30–Apr 12	- Submit final report and project deliverables - Stretch Goal: PR to Lighttpd - Present project	Ensure repo + PDF finalised

Figure 1.1: Project timeline

Chapter 2

State of the Art

This chapter provides an overview of the technologies, standards, and prior work relevant to the development of an Encrypted ClientHello (ECH) test environment for Lighttpd. It first introduces the technical background required to understand ECH within the TLS 1.3 protocol, followed by a review of existing implementations, supporting libraries, and approaches to testing and deployment.

The chapter concludes by identifying limitations in current work and highlighting the research gap addressed by this project.

2.1 Background

2.1.1 Transport Layer Security 1.3

Transport Layer Security (TLS) is the dominant protocol used to secure communication over the Internet. TLS 1.3, standardised in RFC 8446 Rescorla (2018), introduces significant improvements over previous versions, including reduced handshake latency, simplified cryptographic negotiation, and the removal of insecure legacy algorithms.

The TLS 1.3 handshake establishes a secure session between a client and server through a sequence of cryptographic exchanges. During this process, the client initiates communication by sending a *ClientHello* message, which includes supported cipher suites, key exchange parameters, and extensions such as the Server Name Indication (SNI) and Application-Layer Protocol Negotiation (ALPN). The server responds with a *ServerHello*, after which encrypted communication is established.

While TLS 1.3 ensures that application data is encrypted, not all handshake metadata is protected. In particular, the ClientHello message is transmitted in plaintext, exposing fields such as the SNI. This allows network observers to determine the intended destination of a connection, even when the payload is encrypted. As a result, TLS 1.3 does not fully

prevent traffic analysis, censorship, or user profiling based on connection metadata.

This limitation has become increasingly significant as encrypted traffic has grown to dominate Internet communication. The exposure of SNI has been widely identified as a key privacy weakness in the TLS protocol Cloudflare (2018).

2.1.2 Encrypted ClientHello

Encrypted ClientHello (ECH) is a proposed extension to TLS 1.3 that aims to address the privacy limitations associated with plaintext ClientHello metadata. ECH builds upon the earlier Encrypted SNI (ESNI) proposal and extends protection to a broader set of handshake fields IETF TLS Working Group (2023).

ECH introduces a dual-structure ClientHello mechanism consisting of a *ClientHelloOuter* and a *ClientHelloInner*. The ClientHelloInner contains sensitive information such as the true server name and is encrypted using Hybrid Public Key Encryption (HPKE). The ClientHelloOuter contains a minimal set of information required for routing and compatibility, including a public-facing server name.

The encryption of the ClientHelloInner ensures that only the intended server can access sensitive metadata, effectively preventing third parties from observing the destination of the connection. HPKE enables this by combining asymmetric and symmetric cryptographic techniques to securely encapsulate the inner message using a public key provided by the server.

In practice, ECH requires coordination between the client, server, and DNS infrastructure. The server publishes ECH configuration information via DNS records, allowing the client to obtain the necessary public keys prior to initiating the handshake. This introduces additional complexity compared to standard TLS deployment.

Despite these challenges, ECH represents a significant advancement in transport-layer privacy. Early deployments, such as those by Cloudflare, demonstrate the feasibility of ECH in real-world environments Cloudflare (2023). However, the protocol remains under active development, and support across TLS libraries and web servers is still limited.

The experimental nature of ECH highlights the need for reproducible testing environments and detailed evaluation of implementation behaviour. This motivates the work presented in this dissertation, which focuses on enabling and analysing ECH within a lightweight web server context.

2.1.3 TLS Libraries Supporting ECH

The implementation and evaluation of Encrypted ClientHello (ECH) relies heavily on support within TLS libraries. As ECH remains an experimental extension to TLS 1.3,

its availability across widely used libraries is limited and often incomplete. This section outlines the current state of ECH support, with a focus on libraries relevant to this project.

BoringSSL is one of the most mature implementations supporting ECH. Developed and maintained by Google, BoringSSL is a fork of OpenSSL designed for internal use in Google's infrastructure and Chromium-based applications. It includes experimental support for ECH, along with associated tooling such as the `bssl` command-line client. This tool provides detailed insight into TLS handshake behaviour, including ECH negotiation states, making it particularly valuable for debugging and testing purposes.

In contrast, OpenSSL—the most widely used TLS library—has only recently begun to introduce experimental support for ECH, and this support is not yet considered stable or production-ready OpenSSL Project (2023). As a result, many standard web server stacks that rely on OpenSSL, including Apache and NGINX, have limited or no native support for ECH at the time of writing.

Other TLS implementations have also explored ECH integration to varying degrees, though support remains experimental and often limited to specific ecosystems. These implementations are generally less suitable for standalone server deployment or reproducible testing environments compared to BoringSSL.

Due to its relatively advanced support and accessible debugging tools, BoringSSL is commonly used in research and experimental setups involving ECH. Its flexibility allows developers to generate ECH configuration data, inspect handshake traces, and simulate different negotiation scenarios. For this reason, BoringSSL was selected as the primary TLS library for this project.

However, the reliance on experimental libraries introduces additional complexity. Differences in implementation details, configuration formats, and levels of support can lead to interoperability issues between clients and servers. This lack of standardisation further emphasises the need for controlled testing environments and careful validation of observed behaviour.

Overall, while support for ECH is gradually expanding across TLS libraries, it remains fragmented and experimental. This reinforces the importance of the testing framework developed in this work, which provides a structured approach to evaluating ECH behaviour in a controlled setting.

2.1.4 Lighttpd

Lighttpd is a lightweight, open-source web server designed with a strong emphasis on performance, efficiency, and low resource consumption Lighttpd Project (2024). Originally developed to handle high-load environments with minimal overhead, Lighttpd adopts an

event-driven architecture that enables it to efficiently manage large numbers of concurrent connections using a single-threaded model. This design makes it particularly suitable for embedded systems, constrained environments, and experimental research setups.

In contrast to more widely deployed web servers such as Apache and NGINX, Lighttpd prioritises simplicity and modularity. Core functionality is extended through a set of loadable modules, allowing features such as TLS support, URL rewriting, and proxying to be configured as needed. TLS functionality in Lighttpd is typically provided through external libraries, most commonly OpenSSL, via modules such as `mod_openssl`.

The use of external TLS libraries means that support for emerging protocols, such as Encrypted ClientHello (ECH), is dependent on the capabilities of the underlying cryptographic library. As OpenSSL's support for ECH remains experimental and limited, enabling ECH in Lighttpd requires integration with alternative libraries, such as BoringSSL. This introduces additional complexity, including the need to modify build configurations, resolve compatibility issues, and manage ECH-specific key material.

Despite these challenges, Lighttpd presents a valuable platform for experimental work. Its relatively small codebase and modular structure make it easier to modify and adapt compared to larger, more complex web servers. Furthermore, the limited existing research on ECH within lightweight server environments highlights Lighttpd as an appropriate candidate for investigation.

Most existing studies of ECH deployment have focused on large-scale infrastructure and widely used servers such as NGINX, often in conjunction with OpenSSL-based stacks. In contrast, the behaviour of ECH in lightweight, resource-efficient servers remains underexplored. This gap is particularly relevant in contexts where minimal overhead and simplicity are prioritised, such as embedded systems or specialised deployments.

For these reasons, Lighttpd was selected as the target platform for this project. Its architecture enables detailed inspection and control of TLS behaviour, while its lack of established ECH support provides an opportunity to explore integration challenges and evaluate experimental configurations in a controlled environment.

2.2 Closely-Related Work

Encrypted ClientHello (ECH) is an active area of research and development, with much of the existing work focusing on protocol design, large-scale deployment, and integration within widely used web infrastructure.

One of the most significant contributions to the practical deployment of ECH has come from industry, particularly Cloudflare. Early work on Encrypted Server Name Indication (ESNI) demonstrated the feasibility of encrypting the SNI field within the TLS

handshake Cloudflare (2018). This later evolved into ECH, which extends encryption to a broader set of ClientHello parameters. Cloudflare's experimental deployments have shown that ECH can be integrated into real-world systems with acceptable performance overhead, while significantly improving user privacy Cloudflare (2023).

From a standards perspective, the Internet Engineering Task Force (IETF) has played a central role in defining ECH through ongoing draft specifications IETF TLS Working Group (2023). These documents outline the structure of ClientHelloInner and ClientHelloOuter, the use of Hybrid Public Key Encryption (HPKE), and the interaction between TLS and DNS required to support ECH. However, as these specifications are still evolving, implementations often differ in behaviour and level of completeness.

Research into TLS privacy has consistently identified metadata leakage as a key limitation of existing protocols. Prior studies have demonstrated that even when payload data is encrypted, observable features such as SNI can be used for traffic analysis, website fingerprinting, and censorship **?**. These findings have provided strong motivation for the development of ECH and related privacy-enhancing technologies.

In terms of implementation, most existing work has focused on integrating ECH into large-scale web servers and widely used TLS stacks. For example, experimental support has been explored within NGINX and OpenSSL-based environments, often in conjunction with large infrastructure providers. These studies typically emphasise deployment considerations, performance evaluation, and compatibility with existing systems.

However, relatively little work has examined ECH in the context of lightweight web servers or controlled testing environments. In particular, there is a lack of publicly available methodologies for building reproducible ECH test setups that allow detailed inspection of handshake behaviour. This is especially relevant for research and development scenarios where fine-grained control and debugging capabilities are required.

Furthermore, while tools such as BoringSSL provide low-level access to TLS handshake information, there is limited guidance on how these tools can be systematically used to evaluate ECH behaviour across different configurations. As a result, much of the existing work lacks reproducibility and standardised evaluation approaches.

This project addresses these limitations by developing a structured and reproducible test environment for ECH using Lighttpd and BoringSSL. Unlike prior work that focuses on large-scale deployment, this approach enables detailed analysis of handshake behaviour, error conditions, and protocol interactions within a controlled setting.

2.3 Summary

This chapter introduced the technical foundations of TLS 1.3 and ECH, and reviewed existing implementations across a range of TLS libraries. It also examined prior approaches to ECH deployment and testing, highlighting both the potential benefits and practical challenges associated with the protocol.

Existing work has primarily focused on large-scale web servers and general-purpose TLS libraries, with particular emphasis on platforms such as NGINX and OpenSSL-based systems. In contrast, relatively little attention has been given to lightweight, resource-efficient servers such as Lighttpd.

As a result, there is a clear gap in the literature regarding the implementation and evaluation of ECH in lightweight web server environments. This dissertation addresses this gap by designing and implementing a reproducible ECH testing environment specifically for Lighttpd, enabling detailed analysis of its behaviour and limitations.

2.4 Planned Testing and Evaluation

To evaluate the implementation of Encrypted ClientHello (ECH) support within the Lighttpd web server, a number of tests will be conducted. The goal of these tests is to determine whether the implementation functions correctly, maintains compatibility with existing TLS infrastructure, and to measure any performance overhead introduced by ECH.

The evaluation is divided into several categories: functional testing, performance testing, browser-based testing, fallback behaviour testing, and security verification.

2.4.1 Functional Testing

Functional testing will be used to verify that the implementation behaves as expected and that ECH is successfully negotiated between client and server.

The following aspects will be tested:

- Whether a client can successfully establish a TLS connection with the server when ECH is enabled.
- Whether the server correctly processes ECH-enabled ClientHello messages.
- Whether ECH negotiation succeeds when both the client and server support the feature.
- Whether handshake logs confirm that ECH was attempted and accepted.

These tests ensure that the core functionality of the system is working correctly before any further evaluation is performed.

2.4.2 Performance Testing

Performance testing will be conducted to determine whether enabling ECH introduces any measurable overhead during TLS connection establishment.

Two configurations will be compared:

- A baseline configuration where ECH is disabled.
- A configuration where ECH is enabled on the server.

For each configuration, a series of HTTP requests will be sent to the server and timing metrics will be recorded. These metrics include:

- TLS handshake time
- Time to first byte (TTFB)
- Total request completion time

The purpose of this testing is to evaluate whether encrypting the ClientHello message has any noticeable impact on latency or overall connection performance.

2.4.3 Browser-Based Testing

In addition to command-line testing tools, browser-based testing will be conducted to simulate real-world usage scenarios.

A web browser that supports ECH will be configured to connect to the test server. These tests will verify that:

- Web pages hosted on the Lighttpd server load correctly when ECH is enabled.
- The TLS handshake successfully negotiates ECH in a browser environment.
- No errors occur during the connection establishment process.

Automation tools may also be used to repeatedly load pages in order to observe any differences in page load performance.

2.4.4 Fallback Behaviour Testing

ECH is designed to be backwards compatible with existing TLS deployments. If a client or server cannot successfully negotiate ECH, the connection should fall back to standard TLS behaviour.

Tests will therefore be conducted to simulate scenarios where ECH cannot be used, including:

- Clients without ECH support.
- Incorrect or missing ECH configuration.
- Servers rejecting ECH keys.

In these cases, the expected behaviour is that the connection proceeds using standard TLS with plaintext Server Name Indication (SNI). This ensures compatibility with existing web infrastructure.

2.4.5 Security and Privacy Verification

One of the primary motivations behind ECH is to prevent observers from identifying the hostname being accessed during the TLS handshake.

To verify that the hostname is no longer visible in plaintext, network traffic generated during connections will be captured and analysed. The captured packets will be inspected to confirm that:

- The Server Name Indication is not visible in plaintext within the ClientHello message.
- The handshake structure reflects the use of encrypted extensions associated with ECH.

This testing ensures that the implementation successfully achieves the intended privacy improvements provided by ECH.

2.4.6 Test Automation

To improve the reliability and repeatability of the evaluation, automated scripts will be used to run tests repeatedly and collect structured output data.

Automation will allow:

- Running a large number of test requests.
- Collecting consistent timing measurements.
- Comparing results across different configurations.

This approach ensures that the performance analysis is based on a sufficiently large dataset and reduces the impact of variability between individual test runs.

Chapter 3

Implementation

3.1 Testing and Evaluation

To evaluate the implementation of Encrypted ClientHello (ECH) within the Lighttpd web server, a structured series of tests was conducted. The objective of these tests was to verify functional correctness, ensure compatibility with standard TLS behaviour, and assess any performance overhead introduced by ECH.

The evaluation was divided into four primary categories: functional testing, performance testing, fallback behaviour testing, and security verification.

3.1.1 Functional Testing

Functional testing was performed to verify that the system behaved as expected and that ECH negotiation occurred correctly between the client and server.

The following aspects were evaluated:

- Successful establishment of TLS connections when ECH was enabled.
- Correct processing of ECH-enabled *ClientHello* messages by the server.
- Successful negotiation of ECH when both client and server supported the feature.
- Verification of ECH negotiation through handshake logs and client output.

These tests ensured that the core functionality of the implementation operated correctly before further evaluation was conducted.

3.1.2 Performance Testing

Performance testing was conducted to determine whether enabling ECH introduced measurable overhead during TLS connection establishment.

Two configurations were compared:

- A baseline configuration with ECH disabled.
- A configuration with ECH enabled on both client and server.

For each configuration, repeated HTTP requests were issued to the server and timing metrics were recorded. These included:

- TLS handshake time
- Time to First Byte (TTFB)
- Total request completion time

This approach enabled a direct comparison between standard TLS and ECH-enabled connections, allowing the performance impact of ECH to be quantified.

3.1.3 Fallback Behaviour Testing

ECH is designed to remain compatible with existing TLS deployments by falling back to standard TLS when negotiation fails.

Tests were conducted to simulate scenarios where ECH could not be successfully negotiated, including:

- Clients without ECH support.
- Invalid or missing ECH configuration.
- Server-side rejection of ECH parameters.

In these cases, connections were expected to proceed using standard TLS with plaintext Server Name Indication (SNI). These tests verified that compatibility with non-ECH systems was maintained.

3.1.4 Security and Privacy Verification

To evaluate the privacy benefits of ECH, network traffic generated during test connections was captured and analysed.

Packet inspection was used to verify that:

- The Server Name Indication (SNI) was not visible in plaintext within the ClientHello message.
- The handshake structure reflected the use of encrypted ECH extensions.

These checks ensured that the implementation achieved the intended privacy improvements associated with ECH.

3.1.5 Test Automation

To improve reliability and reproducibility, automated scripts were developed to execute tests and collect structured output.

Automation enabled:

- Repeated execution of test cases under consistent conditions.
- Collection of timing data across multiple runs.
- Direct comparison of results between configurations.

This approach reduced variability between individual test runs and ensured that results were based on a sufficiently large dataset.

3.2 Experimental Setup

This section describes the hardware, software, and network configuration used to conduct the experiments. Defining the experimental setup ensures that the methodology is reproducible and that performance results can be interpreted accurately.

3.2.1 Server Environment

The test server consisted of a modified build of the Lighttpd web server with support for Encrypted ClientHello (ECH). TLS functionality was provided using the BoringSSL library, which includes experimental support for ECH.

The server environment included:

- Operating System: Ubuntu Linux
- Web Server: Lighttpd (custom build with ECH support)
- TLS Library: BoringSSL
- Test Content: A static webpage used for repeated HTTP requests

The server was configured to serve HTTPS traffic and provide ECH configuration data required for client-side negotiation.

3.2.2 Client Environment

Testing was performed using command-line tools capable of initiating TLS connections and exposing handshake behaviour.

The client environment included:

- `curl` for HTTP request execution and performance measurement
- BoringSSL's `bssl client` for detailed inspection of TLS handshakes

These tools enabled precise and repeatable testing of both protocol-level behaviour and application-level responses.

3.2.3 Network Configuration

To minimise external variability, testing was conducted within a controlled local environment where the client and server operated on the same network.

This ensured that performance measurements were primarily influenced by TLS configuration rather than network latency or external routing conditions.

Packet capture tools were used where necessary to observe TLS handshakes and validate ECH behaviour.

3.2.4 Test Configurations

Two primary configurations were used:

- **Baseline Configuration:** Standard TLS with ECH disabled
- **ECH Configuration:** TLS with ECH enabled and supported by both client and server

Comparing these configurations allowed the impact of ECH on performance and behaviour to be evaluated.

3.2.5 Data Collection

Test data was collected through automated scripts that repeatedly initiated HTTPS connections to the server. Timing metrics and handshake logs were recorded for analysis.

This dataset formed the basis for evaluating both performance characteristics and ECH negotiation behaviour.

3.3 Evaluation Metrics

To assess the behaviour and performance impact of ECH, a set of metrics was defined. These metrics enabled comparison between standard TLS and ECH-enabled configurations and provided both performance and protocol-level insights.

3.3.1 TLS Handshake Time

TLS handshake time measures the duration required to establish a secure connection between client and server.

Since ECH introduces additional cryptographic operations, this metric was used to determine whether these operations introduced measurable latency.

3.3.2 Time to First Byte (TTFB)

TTFB measures the time between initiating a request and receiving the first byte of the server's response.

This metric reflects the combined impact of TLS negotiation and server processing, providing insight into perceived response time.

3.3.3 Total Request Time

Total request time measures the full duration of an HTTP request, from initiation to completion.

This provides a broader view of performance and helps identify any overall impact of ECH on content delivery.

3.3.4 ECH Negotiation Success

This metric evaluates whether ECH negotiation was successfully completed.

Handshake logs were analysed to determine:

- Whether ECH was attempted

- Whether ECH was accepted or rejected
- Whether the connection completed successfully

3.3.5 Fallback Behaviour

Fallback behaviour was evaluated to ensure that connections reverted to standard TLS when ECH negotiation failed.

This verified compatibility with clients and servers that do not support ECH.

3.3.6 SNI Visibility

To verify privacy improvements, packet captures were analysed to determine whether the Server Name Indication (SNI) was visible in plaintext.

Successful ECH operation was indicated by the absence of plaintext hostname information in the ClientHello message.

3.3.7 Automated ECH Test Suite

In addition to manual testing, a dedicated automated test suite was implemented using Lighttpd's native Perl-based testing framework.

This test suite integrated `curl` and BoringSSL's `bssl client` to validate both HTTP responses and TLS handshake behaviour.

The automated tests covered:

- Standard TLS connections without ECH
- Successful ECH negotiation
- Verification of ECH status via client output
- Rejection of invalid ECH configurations
- Server stability following failed ECH attempts

This automated approach ensured that ECH functionality was consistently validated across multiple test runs in a controlled and reproducible manner.

Chapter 4

Evaluation

This chapter evaluates the implemented Encrypted ClientHello (ECH) test harness and examines its behaviour under a range of controlled configurations. The evaluation focuses on three aspects: correctness of ECH negotiation, compatibility with standard TLS behaviour, and robustness under invalid or malformed inputs. In addition to a matrix-based experimental study, the implementation was also validated using an automated ECH-specific test suite integrated into Lighttpd's native Perl testing framework.

4.1 Experiments

To evaluate the correctness and behavioural properties of the implemented ECH test harness, a full $2 \times 2 \times 2$ experimental matrix was executed. The independent variables were:

- **Server Mode**: ECH enabled (`ech_on`) or ECH disabled (`ech_off`)
- **Client Mode**: ECH-capable client (`ech`) or non-ECH client (`no_ech`)
- **HTTP Mode**: HTTP/1.1 or HTTP/2

This produced eight experimental configurations. Each configuration was executed automatically using the custom `run_matrix.sh` harness.

For each run, the following metrics were recorded:

- TLS handshake outcome (accepted, fallback, or rejected)
- TLS handshake duration (ms)
- HTTP status code

Server Mode	Client Mode	HTTP Mode	Expected Behaviour
ech_on	ech	http1/http2	ECH accepted
ech_on	no_ech	http1/http2	Standard TLS fallback
ech_off	ech	http1/http2	ECH rejected
ech_off	no_ech	http1/http2	Normal TLS

Table 4.1: Experimental matrix used for ECH behavioural validation

- Negotiated HTTP protocol (1.1 or 2)
- Total request time (s)

This design allowed both protocol-level ECH behaviour and application-level HTTP correctness to be evaluated across all combinations of server, client, and protocol mode.

4.2 Results

All eight configurations completed successfully at the HTTP layer. The server started correctly in every case, all requests returned HTTP 200, and both HTTP/1.1 and HTTP/2 negotiation succeeded where expected. No server crashes, hangs, or startup failures were observed during the matrix execution.

4.2.1 ECH Behavioural Outcomes

The observed ECH outcomes matched the expected behaviour for all configurations:

- **ECH accepted:** 2 runs
- **ECH fallback:** 4 runs
- **ECH rejected:** 2 runs

The two acceptance cases occurred when ECH was enabled on the server and the client was ECH-capable. The four fallback cases occurred when the client did not attempt ECH, regardless of server configuration. The two rejection cases occurred when the client attempted ECH while the server had ECH disabled. In these cases, the BoringSSL client reported `ECH_REJECTED`, confirming that the server correctly enforced its configuration.

These results show that the harness correctly distinguishes between acceptance, fallback, and rejection scenarios, and that the observed outcomes are consistent with the intended semantics of ECH and standard TLS interoperability.

4.2.2 Handshake and Request Timing

Representative timing results are shown in Table 4.2.

Configuration	**Handshake (ms)**	**Total Time (s)**
ech_on + ech + http1	77	0.0097
ech_on + no_ech + http1	37	0.0104
ech_off + no_ech + http1	31	0.0164

Table 4.2: Representative handshake and total request timings

The ECH-enabled handshake exhibited a higher handshake duration than the non-ECH cases, which is consistent with the additional cryptographic processing required to construct and process the encrypted *ClientHelloInner*. However, the total request times remained small in all cases, and no configuration showed evidence of application-layer failure or instability.

Given the limited scale of the timing study, these measurements should be interpreted as indicative rather than definitive performance results. Their primary value is to show that ECH can be exercised successfully without disrupting normal HTTPS operation.

Across all runs:

- HTTP success rate: 8/8 (100%)
- Protocol negotiation success: 8/8 (100%)
- Server stability: 8/8 (100%)

Overall, the matrix results confirm that ECH negotiation behaviour was implemented correctly and that HTTP functionality was preserved across all tested configurations.

4.3 Validation Using the Automated ECH Test Suite

In addition to the matrix-based experiments, a dedicated automated ECH test suite was used to validate the correctness and robustness of the implementation.

The test suite was implemented using Lighttpd's native Perl testing framework and extended with ECH-specific test cases. Whereas the experimental matrix focused on a small set of controlled end-to-end combinations, the automated suite exercised a broader range of behaviours, including protocol correctness, application-level routing, and error handling.

At the protocol level, the tests verified that:

- non-ECH clients completed standard TLS handshakes successfully;
- ECH-capable clients successfully negotiated ECH when the server was configured correctly;
- invalid or mismatched ECH configurations were rejected, with the client reporting `ECH_REJECTED`.

At the application level, a specialised configuration using distinct public and hidden virtual hosts was used to validate observable routing behaviour. These tests showed that:

- requests for the hidden hostname without ECH returned public fallback content;
- requests with valid ECH configuration were routed to the hidden virtual host;
- invalid ECH inputs did not expose hidden content.

This is an important result, as it demonstrates that successful ECH negotiation affected not only TLS metadata handling but also server-side routing behaviour. In other words, ECH acceptance was shown to influence the application-level content served by the system.

4.4 Negative and Robustness Testing

The automated test suite also included negative test cases designed to evaluate system robustness under invalid configurations and malformed inputs.

These tests included:

- missing ECH key directories;
- malformed ECH key files;
- truncated ECH configuration data.

In particular, malformed ECH configurations were generated by truncating otherwise valid configuration data by a single byte. Although these inputs remained close in structure to valid ECH data, they were correctly rejected by the server.

Across all negative test cases:

- ECH negotiation failed cleanly without hanging;
- appropriate error messages were produced;

- the server continued to function normally for standard HTTPS requests.

These results demonstrate that the implementation handled erroneous inputs robustly and did not introduce instability into the broader HTTPS service.

4.5 Manual Validation of Test Behaviour

To ensure that the automated tests were validating genuine system behaviour rather than simply confirming script execution, key scenarios were reproduced manually using `curl` and `bssl client`.

Manual validation confirmed that:

- non-ECH connections to the hidden hostname returned public fallback content;
- ECH-enabled connections returned hidden virtual host content;
- invalid ECH configurations were rejected and did not result in successful HTTP responses.

In addition, test fixtures were deliberately modified, for example by swapping public and hidden content, to confirm that the automated tests failed when expected. This provides strong evidence that the test suite validated real protocol and routing behaviour rather than superficial execution success.

4.6 Summary

The evaluation demonstrates that the implemented test harness correctly distinguishes between ECH acceptance, fallback, and rejection scenarios across both HTTP/1.1 and HTTP/2. All configurations completed successfully at the HTTP layer, and no server instability was observed.

The matrix-based experiments confirmed that ECH behaviour matched expectations under all tested combinations of server mode, client capability, and HTTP protocol. The automated ECH test suite further strengthened this result by validating routing behaviour, malformed input handling, and server robustness under failure conditions.

Taken together, these findings show that the implemented framework provides a reliable and reproducible basis for evaluating experimental ECH behaviour in Lighttpd. The work also highlights that ECH can be integrated into a lightweight web server test environment without breaking standard HTTPS functionality.

Chapter 5

Results & Discussion

This chapter presents the key findings of the evaluation and discusses their implications in the context of Encrypted ClientHello (ECH) and secure web communication.

5.1 Discussion of Results

The evaluation demonstrates that the implemented ECH test harness correctly distinguishes between ECH acceptance, fallback, and rejection scenarios across all tested configurations. The experimental matrix confirms that observed behaviour is consistent with the expected semantics of TLS: ECH is accepted only when both client and server support the feature, rejected when the server is not configured for ECH, and otherwise falls back to standard TLS behaviour.

A key finding is that enabling ECH does not negatively impact standard HTTPS functionality. All configurations successfully completed TLS handshakes and returned valid HTTP responses, indicating that ECH can be integrated without breaking compatibility with existing clients or protocols. This aligns with the design goal of ECH as a backwards-compatible extension to TLS and supports observations made in early deployment studies.

From a performance perspective, the results indicate a modest increase in TLS handshake time when ECH is enabled. This overhead is attributable to the additional cryptographic operations required to construct and process the encrypted *ClientHelloInner* message. However, the negligible difference in total request time suggests that this overhead is largely amortised within the overall connection lifecycle. In practical terms, this implies that ECH is unlikely to introduce perceptible latency in typical web interactions, particularly when compared to network-level variability.

A particularly notable result arises from the behavioural testing using separate public and hidden virtual hosts. These tests demonstrate that ECH acceptance enables access

to hidden content, while non-ECH clients are restricted to public fallback content. This confirms that ECH influences observable server behaviour, rather than only affecting TLS handshake metadata.

This finding has broader implications. It suggests that ECH may be used not only as a privacy mechanism to conceal the Server Name Indication (SNI), but also as a mechanism for conditional service exposure. In such a model, sensitive services could be made accessible only to clients capable of negotiating ECH, effectively providing an additional layer of access control at the transport layer. While this extends beyond the original design goals of ECH, it highlights a potential secondary use case that merits further investigation.

The robustness tests further strengthen the evaluation. By introducing malformed and truncated ECH configurations, missing key material, and invalid client inputs, it was shown that the implementation fails safely. In all cases, ECH negotiation was rejected without causing server instability, and standard HTTPS functionality remained unaffected. This indicates that the integration of ECH does not introduce additional risk to system stability, which is a critical consideration for real-world deployment.

Manual validation was also performed to confirm that the automated test suite reflects actual protocol behaviour. By reproducing key scenarios using `curl` and `bssl client`, and deliberately modifying test fixtures, it was verified that the tests detect genuine changes in system behaviour. This provides strong evidence that the evaluation framework measures real protocol-level outcomes rather than superficial execution success.

Overall, the results validate both the correctness of the ECH implementation and the effectiveness of the developed test suite as a reproducible framework for evaluating ECH behaviour in web servers. As shown in Figure 5.1, all test cases completed successfully, including ECH acceptance, fallback behaviour, rejection scenarios, and robustness checks involving malformed and missing configurations. This confirms that the test harness reliably captures real protocol behaviour under a wide range of conditions.

```
seanl@SensDell:~/final-year-project/lighttpd/tests$ ./mod-openssl-ech.t
1..34
ok 1 - generated TLS certificate fixture
ok 2 - generated server ECH fixture
ok 3 - generated mismatched client ECH fixture
ok 4 - generated malformed ECH fixture
ok 5 - Starting lighttpd with mod-openssl-ech.conf
ok 6 - ECH-enabled server serves HTTPS
ok 7 - non-ECH client completed TLS handshake
ok 8 - non-ECH client stayed on standard TLS
ok 9 - non-ECH client received HTTP 200
ok 10 - ECH client completed TLS handshake
ok 11 - valid ECH was accepted
ok 12 - ECH client received HTTP 200
ok 13 - mismatched ECH input was rejected without hanging
ok 14 - mismatched ECH input produced ECH rejection
ok 15 - server stayed healthy after mismatched ECH input
ok 16 - Stopping lighttpd
ok 17 - Starting lighttpd with mod-openssl-tls.conf
ok 18 - non-ECH server serves HTTPS
ok 19 - ECH-capable client was rejected when ECH was disabled
ok 20 - ECH-disabled server returned ECH rejection
ok 21 - non-ECH server stayed healthy after ECH attempt
ok 22 - Stopping lighttpd
ok 23 - starting lighttpd with missing ECH keydir succeeds
ok 24 - missing ECH keydir rejected client without hanging
ok 25 - missing ECH keydir produced ECH rejection
ok 26 - missing ECH keydir was logged
ok 27 - server stayed healthy with missing ECH keydir
ok 28 - Stopping lighttpd
ok 29 - starting lighttpd with malformed ECH key file succeeds
ok 30 - malformed ECH key file rejected client without hanging
ok 31 - malformed ECH key file produced ECH rejection
ok 32 - malformed ECH key file was logged
ok 33 - server stayed healthy with malformed ECH key file
ok 34 - Stopping lighttpd
seanl@SensDell:~/final-year-project/lighttpd/tests$ ./mod-openssl-ech-extra.t
1..30
ok 1 - generated TLS certificate fixture
ok 2 - generated server ECH fixture
ok 3 - generated malformed client ECH config fixture
ok 4 - wrote distinct public and hidden vhost content fixtures
ok 5 - Starting lighttpd with mod-openssl-ech-extra.conf
ok 6 - public vhost baseline served expected content without ECH
ok 7 - ECH fallback: non-ECH client completed TLS handshake
ok 8 - ECH fallback: non-ECH handshake stayed on standard TLS
ok 9 - ECH fallback: non-ECH request returned HTTP 200
ok 10 - ECH fallback: hidden host without ECH served public vhost content
ok 11 - ECH-only host: non-ECH request did not expose hidden content
ok 12 - ECH-only host: ECH client completed TLS handshake
ok 13 - ECH-only host: ECH was accepted
ok 14 - ECH-only host: ECH request returned HTTP 200
ok 15 - ECH-only host: ECH request served hidden vhost content
ok 16 - invalid ECH config was rejected without hanging
ok 17 - invalid ECH config did not complete HTTP 200
ok 18 - invalid ECH config produced a failure trace
ok 19 - server stayed healthy after invalid ECH input
ok 20 - TLS vs ECH comparison: handshake traces differ as expected
ok 21 - sequential ECH connection \#1 stayed consistent
ok 22 - sequential ECH connection \#2 stayed consistent
ok 23 - sequential ECH connection \#3 stayed consistent
ok 24 - sequential ECH connection \#4 stayed consistent
ok 25 - sequential ECH connection \#5 stayed consistent
ok 26 - HTTP/1.1 ECH request completed TLS handshake
ok 27 - HTTP/1.1 ECH request kept ECH enabled
ok 28 - HTTP/1.1 ECH request returned HTTP 200
ok 29 - HTTP/1.1 ECH request served hidden vhost content
ok 30 - Stopping lighttpd
```

Figure 5.1: Execution output from the automated ECH test suite. The results demonstrate successful validation of ECH acceptance, fallback, rejection, and robustness scenarios across multiple configurations.

5.2 Comparison with Existing Work

The findings of this project are broadly consistent with existing work on Encrypted ClientHello (ECH), particularly in relation to its correctness and compatibility with standard TLS behaviour. Prior studies and industry deployments, such as those reported by Cloudflare, have demonstrated that ECH can be integrated into existing infrastructure without breaking HTTPS functionality. The results presented in this work support this observation, as all configurations successfully completed TLS handshakes and returned valid HTTP responses.

However, much of the existing literature focuses on large-scale deployments and integration within widely used web servers such as NGINX. In contrast, this work demonstrates that ECH can also be successfully implemented and evaluated within a lightweight web server environment. This highlights that ECH is not limited to large-scale infrastructure, but can also be explored in more controlled and resource-efficient contexts.

A key distinction between this work and prior research lies in the level of observability and reproducibility. Existing studies often emphasise deployment and performance at scale, but provide limited detail on how ECH behaviour can be systematically tested and validated. The test harness and automated test suite developed in this project address this gap by enabling fine-grained inspection of ECH negotiation, including acceptance, fallback, and rejection scenarios.

Furthermore, while previous work has primarily focused on protocol-level behaviour, this project demonstrates that ECH can influence application-level outcomes, such as virtual host routing. This provides new insight into the practical implications of ECH beyond its intended role as a privacy-enhancing mechanism.

Overall, this work complements existing research by providing a reproducible and controlled framework for evaluating ECH behaviour, while also extending understanding of its impact within lightweight web server environments.

5.3 Limitations

Despite the successful evaluation, several limitations should be acknowledged.

First, the testing environment was limited to a controlled local setup using a modified Lighttpd server and BoringSSL client tools. While this enabled precise control over experimental conditions, it does not fully reflect real-world deployment environments, where factors such as network latency, distributed infrastructure, and heterogeneous client behaviour may influence performance and protocol interaction.

Second, the evaluation focused primarily on correctness and behavioural validation,

with only limited performance analysis. Although the results indicate minimal performance overhead, the scale of testing was relatively small. More comprehensive benchmarking across larger workloads, higher request volumes, and varied network conditions would be required to draw definitive conclusions regarding scalability and production readiness.

Third, while browser-based testing was initially considered, it was not fully implemented within the scope of this project. As a result, evaluation was primarily conducted using command-line tools such as `curl` and `bssl client`. Although these tools provide detailed visibility into TLS behaviour, they do not fully capture the complexity of real-world browser interactions, particularly with respect to DNS-based ECH discovery and client-side optimisations.

Fourth, the implementation relied on manually generated ECH configuration data rather than automated discovery mechanisms such as DNS-based ECH configuration distribution. This simplifies experimentation but does not reflect the full deployment model required for real-world adoption of ECH.

Finally, the evaluation was conducted using a single TLS library (BoringSSL). While this was necessary due to its relatively mature ECH support, differences between TLS implementations may lead to variations in behaviour that were not explored in this work. Broader interoperability testing across multiple TLS stacks would therefore be required to fully validate generalisability.

5.4 Future Work

Several directions for future work arise from this project.

A key extension would be the integration of DNS-based ECH configuration distribution, allowing clients to automatically discover ECH keys. This would enable a more realistic deployment model and provide further insight into interoperability challenges.

Further work could also explore browser-based validation using emerging ECH-capable clients. This would allow evaluation of ECH behaviour in full application stacks and provide a more representative view of real-world usage.

In addition, more comprehensive performance evaluation could be conducted across larger datasets and more complex workloads to assess the scalability of ECH-enabled systems.

Finally, extending the test framework to support additional web servers and TLS libraries would allow for broader interoperability testing and contribute to the standardisation and adoption of ECH.

Chapter 6

Conclusion

This project set out to implement and evaluate Encrypted ClientHello (ECH) support within a Lighttpd web server, with a particular focus on developing a reproducible framework for testing ECH behaviour. The motivation for this work was grounded in the limitations of standard TLS, where sensitive metadata such as the Server Name Indication (SNI) remains visible despite encryption of application data.

The project successfully achieved its primary objectives. ECH functionality was integrated and evaluated within a controlled Lighttpd environment, and a comprehensive test harness was developed to systematically assess ECH behaviour. The evaluation demonstrated that the implementation correctly distinguishes between ECH acceptance, fallback, and rejection scenarios, and that it aligns with the expected semantics of TLS.

A key outcome of the work is the validation that ECH can be deployed without breaking existing HTTPS functionality. All tested configurations successfully completed TLS handshakes and returned valid HTTP responses, confirming the backwards-compatible design of ECH. Furthermore, performance analysis indicated that while ECH introduces a small overhead in handshake time, the impact on overall request latency is negligible in practical terms.

Beyond correctness and performance, the project provides new insight into the behavioural impact of ECH. The use of separate public and hidden virtual hosts demonstrated that ECH can influence application-level outcomes, enabling conditional access to services based on ECH support. This highlights a potential secondary use of ECH as a mechanism for selective service exposure, extending its role beyond privacy enhancement.

The developed test suite represents a significant contribution of this work. By integrating directly with the Lighttpd testing framework, the solution provides a reproducible and extensible method for evaluating ECH behaviour. The inclusion of robustness tests, including malformed inputs and missing configurations, further demonstrates that the implementation fails safely and does not compromise system stability.

However, several limitations remain. The evaluation was conducted in a controlled environment using a single TLS implementation, and did not fully incorporate browser-based testing or DNS-based ECH configuration discovery. As such, while the results provide strong evidence of correctness and feasibility, further work is required to assess real-world deployment scenarios and interoperability across different systems.

Future work could extend this framework by integrating automated ECH configuration distribution, incorporating browser-level validation, and expanding testing across multiple web servers and TLS libraries. Such work would further support the adoption and standardisation of ECH in practical deployments.

In conclusion, this project demonstrates that ECH can be successfully implemented, tested, and evaluated within a lightweight web server environment. It provides both a validated implementation and a reproducible testing framework, contributing to a deeper understanding of ECH behaviour and its implications for secure web communication.

Appendix A

Technical Implementation and Reproducibility Guide

A.1 Overview

This appendix describes the technical implementation of the Encrypted ClientHello (ECH) evaluation environment and provides sufficient detail for the system to be reproduced. It outlines the structure of the codebase, the implementation approach, and the steps required to build, configure, and execute the test environment.

The system consists of two complementary layers: a standalone experimental harness and an integrated regression test suite within Lighttpd. Together, these enable both controlled experimentation and reproducible validation of ECH behaviour.

A.2 Codebase Structure

The implementation is organised across two main components.

The `fyp/` directory contains a standalone experimental harness used for building dependencies, generating ECH material, and running repeatable evaluation experiments. The `lighttpd/tests/` directory contains ECH-specific test cases integrated into Lighttpd's native test framework.

Key files include:

- `mod-openssl-ech.t`: protocol-level ECH tests
- `mod-openssl-ech-extra.t`: behavioural and robustness tests
- `mod-openssl-ech.conf`: ECH-enabled configuration

- `mod-openssl-tls.conf`: TLS-only baseline configuration
- `run_lighttpd.sh`: runtime setup and server execution
- `test_tls.sh`, `test_ech.sh`: standalone validation scripts

A.3 Implementation Approach

ECH support is enabled by compiling Lighttpd's `mod_openssl` module against BoringSSL, which provides experimental ECH functionality. This approach allows the server to expose ECH-specific APIs such as `SSL_CTX_set1_ech_keys` and `SSL_ech_accepted`.

ECH configuration is loaded from a directory of `.ech` files containing both ECH configuration data and associated private key material. These are generated using the `bssl generate-ech` tool and converted into a format compatible with Lighttpd.

During request handling, Lighttpd determines whether ECH was successfully negotiated and applies routing behaviour accordingly. In particular, hidden virtual hosts are only accessible when ECH is accepted, preventing exposure of sensitive hostnames through plaintext SNI.

A.4 Test Harness Design

Two complementary test harnesses are used.

The standalone harness provides a deployment-oriented environment, allowing repeated experiments with structured JSON output. The native Lighttpd test suite provides a regression-oriented environment using TAP output and integrates directly with the existing Lighttpd testing infrastructure.

The test cases are categorised as follows:

- ECH acceptance tests
- fallback behaviour tests
- rejection scenarios
- malformed input tests
- behavioural tests using hidden and public virtual hosts
- sequential and consistency tests

These tests validate both protocol-level correctness and application-level behaviour.

A.5 Execution Flow

The system operates as follows:

1. BoringSSL is built and installed locally.
2. Lighttpd is compiled against the BoringSSL libraries.
3. TLS certificates and ECH configuration material are generated.
4. Lighttpd is launched with an ECH-enabled configuration.
5. Test clients (`curl` and `bssl`) initiate connections.
6. Results are classified and recorded.

A.6 Key Commands

Typical commands used to run the system include:

```
./fyp/scripts/build_boringssl.sh
./fyp/scripts/build_lighttpd.sh
./fyp/scripts/run_lighttpd.sh
./fyp/scripts/test_tls.sh
./fyp/scripts/test_ech.sh
```

To run the integrated Lighttpd test suite:

```
cd lighttpd/tests
./prepare.sh
./mod-openssl-ech.t
./mod-openssl-ech-extra.t
```

A.7 Configuration

The standalone server is configured to run on `127.0.0.1:9443` with TLS enabled. ECH is configured using:

```
ssl.ech-opts = (
  "keydir" => "fyp/run/ech",
  "public-names" => ( "example.com" => "" )
)
```

Hidden virtual hosts are defined using:

```
$HTTP["host"] == "hidden.example.com" {
  ssl.ech-public-name = "example.com"
}
```

This ensures that hidden content is only accessible when ECH is successfully negotiated.

A.8 Design Rationale

Several key design decisions were made:

- The Lighttpd test suite was reused to ensure reproducibility and alignment with upstream testing practices.
- BoringSSL was selected due to its support for ECH and availability of the `bssl` debugging client.
- Separate configurations were used to isolate ECH behaviour from standard TLS.
- Malformed ECH inputs were introduced to validate robustness and safe failure.
- Hidden/public virtual host testing was used to verify privacy-preserving behaviour.

A.9 Failure Handling

The system is designed to fail safely under a range of conditions:

- Missing or invalid ECH configuration results in ECH rejection without affecting TLS.
- Malformed inputs do not crash the server and default to standard HTTPS behaviour.
- Mismatched client configurations are rejected cleanly.

A.10 Reproducibility Guide

To reproduce the environment:

1. Clean previous artefacts:

```
./fyp/scripts/clean.sh
```

2. Build dependencies:

```
./fyp/scripts/build_boringssl.sh
./fyp/scripts/build_lighttpd.sh
```

3. Start the server:

```
./fyp/scripts/run_lighttpd.sh
```

4. Run tests:

```
./fyp/scripts/test_tls.sh
./fyp/scripts/test_ech.sh
```

A.11 Expected Output

Successful execution produces:

- HTTP 200 responses for TLS and ECH-enabled connections
- `Encrypted ClientHello: yes` for successful ECH negotiation
- `Encrypted ClientHello: no` for fallback scenarios
- TAP output indicating passing test cases (e.g. `ok 1, ok 2, ...`)

A.12 Common Issues

Common issues include:

- Incorrect library paths leading to mismatched TLS implementations
- Port conflicts on 9443
- Expired certificates
- Missing dependencies such as `bssl` or `timeout`

Ensuring correct environment configuration resolves the majority of issues.

Appendix B

Repository and Source Code

The full implementation, test harness, and experimental artefacts developed as part of this project are available in the following public repositories.

B.1 Project Repository

The main project repository contains the standalone test harness, configuration files, experimental scripts, and evaluation outputs used throughout this work.

- **Final Year Project Repository:**
  `https://github.com/seanl141/Final-Year-Project-All-Files`

B.2 Modified Lighttpd Source Code

The modifications made to the Lighttpd web server to enable Encrypted ClientHello (ECH) support are provided in a separate fork. This repository contains the adapted `mod_openssl` module and the integrated ECH test cases.

- **Lighttpd ECH Fork:**
  `https://github.com/seanl141/lighttpd1.4`

B.3 Reproducibility

To reproduce the results presented in this dissertation:

1. Clone the Lighttpd fork and build it using the provided scripts.
2. Clone the main project repository.

3. Execute the scripts in the `fyp/scripts/` directory to build dependencies, run the server, and perform tests.

These repositories provide a complete and reproducible environment for evaluating ECH behaviour in a controlled Lighttpd-based setup.

Appendix C

Use of Generative Artificial Intelligence

C.1 Overview

Generative Artificial Intelligence (AI) tools were used during the development of this project as supportive aids for productivity, code understanding, and documentation. These tools were not used to replace core technical work, but rather to assist in refining ideas, improving clarity, and accelerating routine tasks.

All key design decisions, implementation work, experimental setup, and evaluation were carried out independently by the author.

C.2 Tools Used

The following generative AI tools were used:

- **ChatGPT (OpenAI)** – used for explaining concepts, refining technical writing, and assisting with structuring sections of the report.
- **GitHub Copilot / Codex-style tools** – used for minor code suggestions and boilerplate generation.

C.3 Scope of Use

AI tools were used in the following limited and controlled ways:

- **Conceptual clarification:** Explaining aspects of TLS 1.3, ECH, and related cryptographic concepts to support understanding.

- **Code assistance:** Suggesting small code snippets, debugging guidance, and improving script structure.
- **Documentation support:** Refining grammar, improving clarity, and structuring sections such as methodology and appendices.
- **Planning support:** Assisting in outlining test strategies and evaluation approaches.

C.4 Limitations and Verification

All AI-generated outputs were critically evaluated before use. In particular:

- Suggested code was reviewed, tested, and adapted to fit the project requirements.
- Technical explanations were cross-checked against official sources such as RFC 8446 and IETF drafts.
- No AI-generated content was included without verification or modification.

C.5 Academic Integrity

The use of generative AI in this project complies with university guidelines on academic integrity. AI tools were used as assistive technologies rather than sources of original academic contribution.

All substantive work, including system design, implementation of ECH support, development of the test harness, and analysis of results, represents the author's own work.

C.6 Reflection

The use of generative AI tools improved development efficiency, particularly in debugging and structuring documentation. However, their use required careful validation, especially when dealing with security-sensitive protocols such as TLS and ECH.

This highlights the importance of combining AI-assisted development with strong domain knowledge and critical evaluation.

Bibliography

Cloudflare (2018). Encrypting sni: Fixing one of the core internet privacy problems. https://blog.cloudflare.com/esni/.

Cloudflare (2023). Encrypted client hello (ech): The future of tls privacy. https://blog.cloudflare.com/encrypted-client-hello/.

IETF TLS Working Group (2023). Tls encrypted clienthello. https://datatracker.ietf.org/doc/draft-ietf-tls-esni/.

Lighttpd Project (2024). Lighttpd documentation. https://redmine.lighttpd.net/projects/lighttpd/wiki.

OpenSSL Project (2023). Openssl ech support discussion. https://github.com/openssl/openssl/issues.

Rescorla, E. (2018). The transport layer security (tls) protocol version 1.3. https://datatracker.ietf.org/doc/html/rfc8446.